Day-1

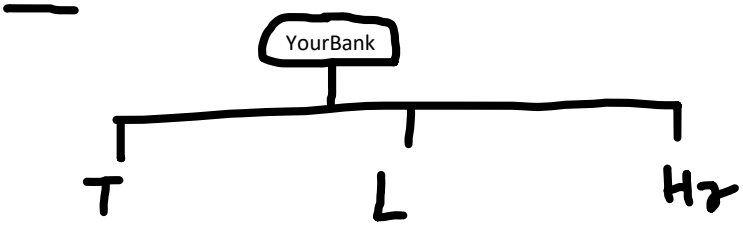
Wednesday, September 17, 2025 1:28 PM

It's an open source relational database management system to store the data in the Form of tables (Rows & Columns)

Banking
E-shopping
ERP
Retail(Inventory management)

Database
Tables
Rows
Columns
Primary Key -- UNIQUE VALUES YOU WANT TO STORE

Database --- is a collection of multiple tables of its similar type of database



Create database <dbname>

Use <dbname> to select the database on which you have to work on

CREATE DATABASE EMS;

EMP_ID	NAME	DEPT	SALARY
101	NITI	TRAINING	45000

CREATE TABLE EMPLOYEES(EMP_ID INT PRIMARY KEY, NAME VARCHAR(50), DEPT VARCHAR(30), SALARY INT);

```
mysql> USE DEMO_DB;
Database changed
mysql> USE EMS;
Database changed
mysql> CREATE TABLE EMPLOYEES(EMP_ID INT PRIMARY KEY, NAME VARCHAR(50), DEPT VARCHAR(30), SALARY INT);
Query OK, 0 rows affected (0.03 sec)

mysql> SELECT * FROM EMPLOYEES;
Empty set (0.01 sec)

mysql> DESC EMPLOYEES;
+--------+-------------+------+-----+---------+-------+
| Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| EMP_ID | int         | NO   | PRI | NULL    |       |
| NAME   | varchar(50) | YES  |     | NULL    |       |
| DEPT   | varchar(30) | YES  |     | NULL    |       |
| SALARY | int         | YES  |     | NULL    |       |
+--------+-------------+------+-----+---------+-------+
4 rows in set (0.03 sec)

mysql> SHOW TABLES;
+---------------+
| Tables_in_ems |
+---------------+
| employees     |
+---------------+
1 row in set (0.01 sec)

mysql>
```

```
mysql> INSERT INTO EMPLOYEES VALUES(101,"NITI DWIVEDI","TRAINING",45000);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM EMPLOYEES;
+--------+--------------+----------+--------+
| EMP_ID | NAME         | DEPT     | SALARY |
+--------+--------------+----------+--------+
|    101 | NITI DWIVEDI | TRAINING |  45000 |
+--------+--------------+----------+--------+
1 row in set (0.00 sec)
```

INSERT INTO <TABLENAME> VALUES() --- USE TO INSERT THE VALUES IN A TABLE
SELECT * FROM <TABLENAME> -- USE TO VIEW THE RECORDS

Ems -- YOU WANT TO CREATE , UPDATE , DELETE AND MODIFY THE DATA SO PERFORMING THESE OPERATIONS
WE HAVE COMMANDS AS BELOW:

CREATE --- INSERT INTO
VIEW / READ -- SELECT * FROM
MODIFY -- UPDATE <TABLE NAME> SET <NEW DATA> WHERE <CONDITION>
DELETE --- DELETE FROM <TABLE NAME> WHERE <CONDITION>

ACTIVITY : PERFORM THE DATA BASE AND CREATE A TABLE AS PRODUT AND ADD RECORDS IN IT
AND THEN SHOW ALL PRODUCTS

```
mysql> SELECT * FROM EMPLOYEES;
+--------+--------------+----------+--------+
| EMP_ID | NAME         | DEPT     | SALARY |
+--------+--------------+----------+--------+
|    101 | NITI DWIVEDI | TRAINING |  45000 |
|    102 | JATIN        | TRAINING |  45000 |
|    103 | JIYA         | TRAINING |  55000 |
+--------+--------------+----------+--------+
3 rows in set (0.00 sec)

mysql> UPDATE EMPLOYEES SET DEPT="OPERATIONS"  WHERE EMP_ID=103
    -> ;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM EMPLOYEES;
+--------+--------------+------------+--------+
| EMP_ID | NAME         | DEPT       | SALARY |
+--------+--------------+------------+--------+
|    101 | NITI DWIVEDI | TRAINING   |  45000 |
|    102 | JATIN        | TRAINING   |  45000 |
|    103 | JIYA         | OPERATIONS |  55000 |
+--------+--------------+------------+--------+
3 rows in set (0.00 sec)

mysql> DELETE FROM EMPLOYEES WHERE EMP_ID=103
    -> ;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM EMPLOYEES;
+--------+--------------+----------+--------+
| EMP_ID | NAME         | DEPT     | SALARY |
+--------+--------------+----------+--------+
|    101 | NITI DWIVEDI | TRAINING |  45000 |
|    102 | JATIN        | TRAINING |  45000 |
+--------+--------------+----------+--------+
2 rows in set (0.00 sec)
```

MAIN CATEGORIES OF COMMANDS

DDL : DATA DEFINITION LANGUAGE (When it is related to your table)
Create , Alter , Drop and Truncate
CREATE TABLE EMPLOYEES(EMP_ID INT PRIMARY KEY, NAME VARCHAR(50), DEPT VARCHAR(30), SALARY INT);
Alter table employees add email varchar(30);
Drop table employees;
Truncate employees; Use to delete everything in the table without having any condition

DML : DATA MANIPULATION LANGUAGE (When it is related to your table data (rows)
Insert , Update , Delete
INSERT INTO EMPLOYEES VALUES(103,"JIYA","TRAINING",55000);
UPDATE EMPLOYEES SET DEPT="OPERATIONS" WHERE EMP_ID=103;
DELETE FROM EMPLOYEES WHERE EMP_ID=103; use to delete a single or more than one record based on condition

DQL : DATA QUERY LANGUAGE (Performing the queries on your data)
Select
Select * from employee;
Select * from employees where salary>5000;
Select * from employees order by salary desc;

TCL : TRANSACTION CONTROL LANGUAGE
to ensure the data consistency using transactions
Commit , Rollback , savepoint

DCL : Data Control Language
Grant , Revoke

Some of the SQL Select queries

```
select * from employees;
 select 1+1 as addition;
select now();
select concat("new", "delhi")as city;
 select * from employees order by emp_id desc;
 select * from employees order by emp_id desc limit 1;
```

The query gets evaluated using
From ----> select ----> orderby

1- Let's say if you want to display the last name in descending order and then the first name in ascending

Niti dwivedi
Komal sharma
Jiya dwivedi
Jatin dwivedi
Lovely sharma

```
 select * from employees order by lastname desc firstname asc;
```

Dwivedi Jatin
Dwivedi Jiya
Dwivedi Niti
Sharma Komal
Sharma Lovely

2- To display the total salary after increment 1000 as a bonus

```
 Select  emp_id ,name,dept ,salary+1000 as totalbonusedsal from employees order by totalbonusedsal;
 Select  emp_id ,name,dept ,salary+1000 as totalbonusedsal from employees order by totalbonusedsal desc limit 1;
```

3 - display the salaries of all employees whose department is not in training and operations

```
select name ,salary , dept from employees where dept not in('training','admin');
```

4. display the salaries of all employees whose department is unique

```
select distinct dept from employees;
```

Don’t use if you want to display distinct depart because in this case name and salary are not same so if name and salary is same then it will work

```
select distinct dept ,name, salary from employees where dept in('training','admin');
```

```
mysql> select * from employees;
+--------+--------------+------------+--------+------------------+
| EMP_ID | NAME         | DEPT       | SALARY | email            |
+--------+--------------+------------+--------+------------------+
|    101 | NITI DWIVEDI | TRAINING   |  45000 | NULL             |
|    102 | JATIN        | TRAINING   |  50000 | NULL             |
|    103 | JIYA         | OPERATIONS |  55000 | jiya@gmail.com   |
|    104 | Simran       | admin      |  55000 | simran@gmail.com |
|    105 | Simran       | admin      |  55000 | simran@gmail.com |
+--------+--------------+------------+--------+------------------+
```

5 - display the count of unique departments

And may also count the unique department we have below query

```
select count(distinct dept)  from employees;
```

6 - display the data where department is training or salary is greater than 25000

```
select * from employees where dept = 'training' or salary>=25000;
```

7. Like operator to get the name starts with or ends with using wild card characters % . __(underscore)

```
select name ,emp_id, dept,salary from employees where name like "____%N";
select name ,emp_id, dept,salary from employees where name like "%di";
select name ,emp_id, dept,salary from employees where name like "NI%";
select name ,emp_id, dept,salary from employees where name like "____n%";
```

Insert

```
create table department (dept_id int auto_increment primary key , dept_name varchar(20));
 insert into department (dept_name) values("OPERATIONS");
```

```
select last_insert_id();  // to check the last incremented value;
truncate table department;   // to delete the entire data
```

Alter commands

```
create table department ( dept_id int primary key , emp_id int not null, foreign key(emp_id) references employees(emp_id));
 alter table department add s_no int auto_increment primary key;
// to change a table name
rename table department to hr_department;

// add a new column name
alter table department add column dept_name varchar(30) after dept_id;

// drop any column from a table
 alter table employees drop column email;

// rename the column name
alter table employees rename column dept to deptname;

// modifying the width or datatype of a column
 alter table employees modify column deptname varchar(100);
```

Constraints :

Different types of constraints we can apply on a table columns which enforcing the data validation

Types of constraints are below:

Default
Unique
Not Null
Check

```
Create table cart_items(item_id int auto_increment primary key,
name varchar(255) not null,quantity int not null,
price dec(5,2) not null,
sales_tax dec(5,2) not null default 0.1,
check(quantity> 0),
check (sales_tax >=0));
```

Day-2

28 January 2026 09:00

Joins : They are used to combine rows from two or more tables based on related column between Them.

Types of Joins

1) Inner join --
2) Left Join -- (Left Outer join
3) Right Join -- (Right Outer Join)
4) Full Join -- (Full Outer Join)
5) Self Join
6) Cross Join

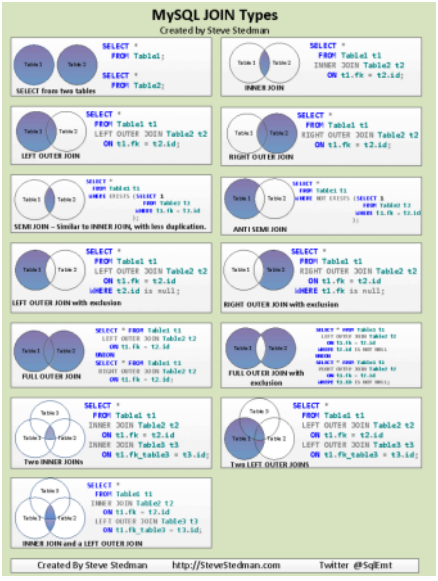


Table - 1 (Left) Table - 2(Right)

```
mysql> select * from employees;
+-------------+---------+---------------+
| employee_id | name    | department_id |
+-------------+---------+---------------+
|         101 | Jiya    |            10 |
|         102 | Shubham |            20 |
|         103 | Rohit   |            30 |
|         104 | Richa   |            10 |
+-------------+---------+---------------+
```

```
create table department (department_id int primary key , department_name varchar(100));
create table employees (employee_id int primary key , name varchar(100), department_id int , foreign key(department_id) references department(department_id));

insert into department values(10,"HR") ,(20,"IT") , (30,"Sales");
insert into employees values(101,"Jiya" ,10),(102,"Shubham" ,20),(103,"Rohit" ,30),(104,"Richa" ,10),(40,"Admin") ,(50,"Training");
```

Select columns from table1 <alias name> inner join table 2 <alias name> on table1.fk = table2.pk

1. Inner Join : An inner Join returns only the row h tables.
   If there is no match , then no row is returned .

   select e.name , d.department_name from em epartment_id = d.department_id;

```
mysql> select * from department;
+---------------+-----------------+
| department_id | department_name |
+---------------+-----------------+
|            10 | HR              |
|            20 | IT              |
|            30 | Sales           |
|            40 | Admin           |
|            50 | Training        |
+---------------+-----------------+
```

```
+---------+-----------------+
| name    | department_name |
+---------+-----------------+
| Jiya    | HR              |
| Shubham | IT              |
| Rohit   | Sales           |
| Richa   | HR              |
+---------+-----------------+
4 rows in set (0.00 sec)
```

2. Right Join : It gives all records of right table i.e. 2 (department)and matching records of table 1(employees)

   select e.name , d.department_name from employees e right join department d on e.department_id = d.department_id;

```
+---------+-----------------+
| name    | department_name |
+---------+-----------------+
| Jiya    | HR              |
| Richa   | HR              |
| Shubham | IT              |
| Rohit   | Sales           |
| NULL    | Admin           |
| NULL    | Training        |
+---------+-----------------+
6 rows in set (0.00 sec)
```

3. Left Join : It gives all records of Left table i.e. 1(employees) and matching records of table 2(department)

```
+---------+-----------------+
| name    | department_name |
+---------+-----------------+
| Jiya    | HR              |
| Shubham | IT              |
| Rohit   | Sales           |
| Richa   | HR              |
+---------+-----------------+
4 rows in set (0.00 sec)
```

4. A full join returns all rows when there is a match in either the left or right table. When there's not match , null is returned for the non-matching side.

Note* - MySQL does not support FULL Outer Join directly , but you can do this with Union.

select e.name, d.department_name from employees e left join department d on e.department_id = d.department_id union select ename ,d.department_name from employees e right join department d on e.department_id = d.department_id;

```
+---------+-----------------+
| name    | department_name |
+---------+-----------------+
| Jiya    | HR              |
| Shubham | IT              |
| Rohit   | Sales           |
| Richa   | HR              |
| NULL    | Admin           |
| NULL    | Training        |
+---------+-----------------+
6 rows in set (0.04 sec)
```

5. Cross Join : will return the cartesian product of the two tables, means it combines every row from the first table With every row from the second table . It does not require ON clause

Color	Size
Red	Small
Green	Medium
Blue	Large

6. Self Join : When a table is joined with itself . It is useful for hierarchical or adjacency data structures, such as organizational chart where employees have managers.

Select a.column,b.column from table a inner join table b on a.column = b.column
create table empmanager (emp_id int , name varchar(50) , manager_id int);

insert into empmanager (emp_id,name)values(1, "Niti");
insert into empmanager values(2, "jiya" , 1),(3, "Shubham" , 2),(4, "Richa" , 1);

Employee_id	Name	Managerid
1	Niti	Null
2	Jiya	1
3	Shubham	2
4.	Richa	1

Output of Cross Join

Color	Size
Red	Small
Red	Medium
Red	Large
Green	Small
Green	Medium
Green	Large
Blue	Small
Blue	Medium
Blue	Large

Select e1.name as employeename, e2.name as managername from empmanager e1 left join empmanager e2 on e1.manager_id = e2.emp_id;

```
+--------------+-------------+
| employeename | managername |
+--------------+-------------+
| Niti         | NULL        |
| jiya         | Niti        |
| Shubham      | jiya        |
| Richa        | Niti        |
+--------------+-------------+
```

```
mysql> select * from empmanager;
+--------+---------+------------+
| emp_id | name    | manager_id |
+--------+---------+------------+
|      1 | Niti    |       NULL |
|      2 | jiya    |          1 |
|      3 | Shubham |          2 |
|      4 | Richa   |          1 |
+--------+---------+------------+
4 rows in set (0.00 sec)
```

Subquery : Nested (select * from employees w ept_id=101))

In SQL, a **subquery** (or inner query) is a query nested within another statement. The primary distinction between a standard subquery and a **correlated subquery** lies in their dependency on the outer query and how many times they execute.

Subquery (Non-Correlated)

A standard subquery is independent. It executes **once** before the outer query runs, and its result is then passed to the outer query.

- **Execution:** Runs once, independently.
- **Dependency:** The outer query depends on the inner query's result.
- **Example:** Finding all employees in a specific department name.

```
SELECT * FROM Employees
WHERE DepartmentID = (SELECT DepartmentID FROM Departments WHERE Name = 'Sales');
```

- ## Correlated Subquery

  A correlated subquery is dependent on the outer query. It references one or more columns from the outer query, meaning it must be re-evaluated **for every row** processed by the outer query.
- **Execution:** Runs once for each row of the outer query.

- **Dependency:** The inner query depends on values from the outer query.
- **Performance:** Generally slower than non-correlated subqueries because of the repeated execution.
- **Example:** Finding employees who earn more than the average salary *of their own department*.

Empid	Empname	Deptname	Salary
1	Dff	Sales	354
2	Df	Hr	54
3		Sales	3546
		Hr	453
		Sales	35
		Admin	3
		Hr	5
		Admin	546

Empid	Empname	Deptname
1		Sales
3		Sales
5		Sales

5000

```
SELECT Name, Salary
FROM Employees e1
WHERE Salary > (SELECT AVG(Salary) FROM Employees e2 WHERE e1.DeptID = e2.DeptID);
```

Empid	Empname	Deptname
1		Hr

```
mysql> select * from departments;
+---------+-------------+
| dept_id | dept_name   |
+---------+-------------+
|      10 | HR          |
|      20 | IT          |
|      30 | Sales       |
|      40 | Engineering |
|      50 | Marketing   |
+---------+-------------+
5 rows in set (0.00 sec)

mysql> select * from employees
    -> ;
+--------+---------+-------+--------+---------+
| emp_id | fname   | lname | salary | dept_id |
+--------+---------+-------+--------+---------+
|    101 | Jiya    | D     |  60000 |      10 |
|    102 | Shubham | S     |  50000 |      20 |
|    103 | Rohit   | K     |  67700 |      30 |
|    104 | Richa   | K     |  40000 |      10 |
+--------+---------+-------+--------+---------+
4 rows in set (0.00 sec)
```

```
create table employees(emp_id int, fname varchar(40) , lname varchar(30) ,salary int , dept_id int);
 create table departments(dept_id int ,dept_name varchar(50));

insert into employees values(101,"Jiya" ,"D", 60000 , 10),(102,"Shubham", "S" , 50000 , 20),(103,"Rohit" , "K" , 67700,30),(104,"Richa" ,"K" , 40000 , 10);
insert into departments values(10,"HR") ,(20,"IT") , (30,"Sales"),(40,"Engineering"), (50,"Marketing");
```

Scalar Subquery : It returns exactly one column and one row -- single value -- like average

```
mysql> select avg(salary) from employees;
+--------------+
| avg(salary) |
+--------------+
|  54425.0000 |
+--------------+
```

```
select fname , lname from employees where salary>(select avg(salary) from employees);
+--------+--------+---------+
| fname | lname | salary |
+--------+--------+---------+
| Jiya  | D      |  60000 |
| Rohit | K      |  67700 |
+--------+--------+---------+
2 rows in set (0.00 sec)
```

Multi Row -- where it will return single column with multiple rows (like a list) that we can achieve using
IN, NOTIN, ANY ,ALL

```
mysql> select dept_id from departments where dept_name in ('Sales' , 'Engineering');
+---------+
| dept_id |
+---------+
|      30 |
|      40 |
+---------+
```

For eg : select fname, lname from employees where dept_id in (select deptid from departments where dept_name in ('Sales' , 'Engineering'));

```
mysql> select fname, lname from employees where dept_id in(select dept_id from departments where dept_name in ('Sales' , 'Engineering'));
+-------+-------+
| fname | lname |
+-------+-------+
| Rohit | K     |
+-------+-------+
1 row in set (0.01 sec)
```

Q1 .select all from departments where no employees are assigned to them.

```
select *
from departments
where dept_id NOT IN (
    select dept_id from employees
);
```

Q2. find all employees whose salary is greater than any salary in the sales department

Select * from employees where salary> any(select salary from employees where dept_id=2);
Select * from employees where salary > any(3434,5757);

What is query optimization ??

Writing sql(and designing tables) so the databases returns result faster, using less cpu , memory and disk i/o

database will be slow mainly because of below resons:

a) Full table scans
b) Bad Joins
c) Missing or wrong indexes
d) Fetching unnecessary data
e) Poor filtering

Optimization = Help the query to do the less work

select * from employees where dept_id=20;
,
Think you have 10 millions of records
First it will read all the columns (Select) ,
No index is defined on department_id
Database performs full table scan and then reads 10 millions rows

Indexing are of two types clustered and non -clustered
Select emp_id,name, salary from employees where dept_id=20;

Index will help like a book index , so instead of every page we can directly jump to a relevant rows

```
SELECT
   INDEX_NAME,
   COLUMN_NAME,
   TABLE_NAME,
   TABLE_SCHEMA
FROM
   INFORMATION_SCHEMA.STATISTICS
WHERE
   TABLE_NAME =employees
   AND TABLE_SCHEMA = ems;
```

Day-3

29 January 2026 09:03

Normalization ; when we have a large dataset in a single table which is not feasible to read the proper data Because multiple columns may or may not belong to each other so Normalization is core database design

It is a process of decomposing large tables into small tables and columns are also related to each other.

If data is not normalized :

1- Data redundancy
2- Update anomalies
3- Insert anomalies
4- Delete anomalies

So normalization is to organize data to reduce duplication and maintain integrity

A company stores customer orders details in One table

Orderdata

Orderid	Customer_name	Customer_phone	Product_names	Product_prices	Order_date

Problems are :
Multiple values in one column
Hard to query individual products
Data consistency risk

That's why Normalized data can be done by following the rules of 1NF , 2NF ,3 NF …..

1NF say's

Each column has single value (atomic values)
No repeating group
Each row uniquely identifiable

Below is not in 1NF

Orderid	Customer_name	Customer_phone	Product_names	Product_prices	Order_date
101	Amit	945453443	Laptop, mouse	70000,500	2026-01-29
102	Rahul	887798798	Keyboard	1200	2026-01-12

After 1NF

Orderid	Customer_name	Customer_phone	Product_name	Product_prices	Order_date
101	Amit	945453443	Laptop	70000	2026-01-15
101	Amit	945453443	mouse	500	2026-01-15

102	Rahul	887798798	Keyboard	1200	2026-01-12

2NF says

It should be in 1 NF first
No partial dependency
(Composite key) combination of two keys to make a primary key (name+phone)

Composite key let say is (order_id , product_name)
Customer_name depends only on order_id
Customer_phone depends only on order_id
Product_price depends only on product_name
Order_date depends only on order_id

Violation of 2 NF

Orders

Order_id	Customer_name	Customer_phone no	Order_date
101	Amit	687987987	2026-01-15
102	Rahul	343434534	2026-01-12

Order_items

Order_id	Product_name	Product_price
101	Laptop	70000
101	Mouse	500
102	Keyboard	1200

3NF says

It should be in 2nf
No transitive dependency

- **Definition:** If
  $A \rightarrow B, B \rightarrow C$
  , then $A \rightarrow C$
  which is a transitive dependency. This usually happens in tables with three or more attributes.
- **Example:**
  In a Studenttable, Student_ID(Key) →Batch_ID→Batch_Name. The Batch_Namedepends on Batch_ID
  , which depends on Student_ID

- **Normalization:** To achieve Third Normal Form (3NF), transitive dependencies must be removed to eliminate insertion, update, and deletion anomalies.

To remove transitive dependency

Customer table

Customer_id	Customer_name	Customer_phone no
C1	Amit	687987987

C2	Rahul	343434534

Orders

Order_id	Customer_id	Order_date
101	C1	2026-01-15
102	C2	2026-01-12

Product table

product_id	Product_name	Product_price
P1	Laptop	70000
P2	Mouse	500
P3	Keyboard	1200

Order_items

Order_id	Product_id
101	P1
101	P2
102	P3

BCNF -- Sometimee 3NF is also not enough then we have to go for BCNF also

A table is in BCNF if

Every functional dependency A->B
Where A must be the super key and generally BCNF eliminates functional dependency anomalies

In a college data

Student_id	Course	Trainer
S1	Java	Niti
S2	Java	Niti
S3	React	Jiya
S4	Reat	Jiya

Student_id , course is taught by trainer
Course -> trainer

Course	Trainer
Java	Niti
React	Jiya

Student_id	Course
S1	Java
S2	Java
S3	React
S4	Reat

Appointment_Id	Patient_name	Patient_phone	Doctor_specialization	Room_no.	Appointment_date

Mongo DB - NoSQL Database

Not Only SQL . Main advantages of this database are below:

1) Huge data volume
2) High Traffic
3) Semi structured / unstructured data

4)Fast reads/write
In some cases mySQL struggled in modern systems
Fixed schema
Costly joins
Vertical scaling(expensive)
Poor fit for JSON /API driven apps

```
"id":101,
"name":"Niti Dwivedi",
"courses":["Java","React"]
```

Mysql	NoSql
Database	Database
Table	Collection
Row	Document
Column	Field
Join	Reference/ Embedded

Aggregate Pipeline:

$match : Filter
$project : to select / transform the fields
$group : Aggregate
$sort : sorting
$lookup : Joins